Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

[CC3069] Computación Paralela y Distribuida

Proyecto 2

Búsqueda de claves AES-128 con Open MPI y OpenMP

Nombre	Carné
Dulce Ambrosio	231143
Daniel Chet	231177
Cristian Túnchez	231359

Índice

1. Introducción **3**
2. Antecedentes **3**
2.1 AES-128 y modo CBC 3
2.2 Búsqueda por fuerza bruta 3
2.3 Open MPI 3
2.4 OpenMP 4
2.5 Speedup y eficiencia 4
3. Entorno de trabajo **4**
4. Inciso 1: cifrado y descifrado de archivos **5**
4.1 Conversión de la clave a 128 bits 5
4.2 Formato del archivo cifrado 5
4.3 Evidencia 5
5. Inciso 2: búsqueda de la clave con 4 procesos **6**
5.1 Programa base: estrategia cíclica 6
5.2 Resultados 6
5.3 Inciso 2d: ¿por qué varía el tiempo según la clave? 7
6. Inciso 3: estrategia alternativa híbrida MPI + OpenMP **7**
6.1 Descripción 7
6.2 Justificación 7
6.3 Comparación de tiempos 8
6.4 Tasa de prueba y speedup 9
6.5 Ventajas y limitaciones 9
7. Inciso 4: funcionamiento de las estrategias **10**
7.1 Reparto de las claves 10
7.2 Diagramas de flujo 11
7.3 Pseudocódigo de la estrategia híbrida 13
7.4 Comunicación cuando se encuentra la clave 13
8. Discusión **13**
8.1 Retos y soluciones 13
8.2 Lo aprendido 14
9. Conclusiones **14**
10. Recomendaciones **14**
11. Referencias **15**
Anexo 1. Catálogo de funciones y librerías **16**
A1.1 Funciones del programa 16
A1.2 Librerías y rutinas externas 17
Anexo 2. Bitácora de pruebas y speedups **18**
A2.1 Pruebas automáticas 18
A2.2 Corridas individuales 18
A2.3 Estimaciones de las búsquedas detenidas 19
A2.4 Comandos y salidas registradas 20

1. Introducción

AES-128 es un cifrado simétrico por bloques. Quien conoce la clave descifra al instante; quien no la conoce solo puede probar claves una por una (fuerza bruta). En este proyecto se partió de los programas del Laboratorio 03 para cifrar y descifrar archivos de texto con AES-128 y para recuperar la clave de un mensaje cifrado probando candidatas hasta que el texto descifrado contiene la frase conocida *es una prueba de*.

La búsqueda se implementó en versión secuencial y en versión paralela con Open MPI. La versión MPI del laboratorio reparte bloques de claves por turnos entre los procesos (estrategia cíclica). Como estrategia alternativa se propuso e implementó una estrategia **híbrida MPI + OpenMP**: los procesos MPI se reparten tramos contiguos del rango y, dentro de cada proceso, un equipo de hilos OpenMP se divide cada bloque de claves.

El informe presenta los antecedentes, el desarrollo de cada inciso, los diagramas de ambas estrategias, las mediciones de tiempo y *speedup* con 4 núcleos, y las conclusiones. Los anexos contienen el catálogo de funciones y la bitácora de pruebas con sus registros.

2. Antecedentes

2.1 AES-128 y modo CBC

AES (*Advanced Encryption Standard*) es el cifrado por bloques estandarizado por el NIST en FIPS 197 [1]. Opera sobre bloques de 128 bits (16 bytes) y admite claves de 128, 192 o 256 bits; AES-128 usa una clave de 16 bytes y 10 rondas de sustitución, permutación, mezcla de columnas y suma de la subclave de ronda.

Para cifrar mensajes más largos que un bloque se usa un modo de operación. En el modo CBC (*Cipher Block Chaining*) cada bloque de texto plano se combina con XOR con el bloque cifrado anterior antes de cifrarlo; el primer bloque se combina con un vector de inicialización (IV) aleatorio. Como el mensaje no siempre mide un múltiplo de 16 bytes, se agrega relleno PKCS#7: *n* bytes con valor *n*. Al descifrar con una clave equivocada, el relleno casi siempre resulta inválido, lo que permite descartar la mayoría de candidatas rápidamente.

2.2 Búsqueda por fuerza bruta

Un ataque de fuerza bruta prueba todas las claves posibles hasta encontrar la que produce un texto reconocible. Su costo crece exponencialmente con el número de bits: probar 2^{56} claves requiere $2^{56} \approx 7.2 \times 10^{16}$ descifrados en el peor caso. Por eso el tiempo depende de dos factores: cuántas claves por segundo se prueban y en qué posición del recorrido está la clave correcta. Paralelizar aumenta lo primero; la estrategia de distribución decide lo segundo.

2.3 Open MPI

MPI (*Message Passing Interface*) es el estándar de programación paralela con memoria distribuida. Un programa MPI se ejecuta como varios procesos independientes (*ranks*) que se comunican mediante mensajes; pueden estar en la misma máquina o en varias.

Open MPI es una implementación libre del estándar. En este proyecto se usan operaciones colectivas: `MPI_Bcast` envía el archivo cifrado desde el rank 0 a todos los procesos, `MPI_Allreduce` combina el resultado de cada ronda (se toma el mínimo) y lo entrega a todos, y `MPI_Reduce` junta los conteos y tiempos finales en el rank 0.

2.4 OpenMP

OpenMP es una interfaz de programación paralela con memoria compartida basada en directivas del compilador. La directiva `#pragma omp parallel for` divide las iteraciones de un ciclo entre un equipo de hilos; la cláusula `schedule(dynamic, n)` entrega porciones de *n* iteraciones a los hilos conforme se van desocupando, y `#pragma omp atomic` actualiza una variable compartida sin condiciones de carrera. Combinar MPI entre procesos y OpenMP dentro de cada proceso es el modelo híbrido habitual en clústeres de nodos multinúcleo. Para que los hilos convivan con MPI, el programa se inicia con `MPI_Init_thread`; en el nivel `MPI_THREAD_FUNNELED` solo el hilo principal puede llamar a MPI.

2.5 Speedup y eficiencia

El *speedup* compara el tiempo secuencial con el paralelo: $S = \frac{Tsecuencial}{Tparalelo}$. La eficiencia divide el *speedup* entre el número de núcleos: $E = \frac{S}{p}$. Con p = 4, el valor ideal es S = 4 y E = 1. En una búsqueda que se detiene al encontrar la clave, cambiar el orden de exploración puede dar *speedups* mayores que p (superlineales) o menores que 1, porque cada estrategia prueba una cantidad distinta de claves antes de llegar a la correcta.

3. Entorno de trabajo

Componente	Versión o valor
Equipo	Intel Core i9-13980HX (24 núcleos, 32 hilos lógicos), WSL 2
Sistema	Linux-6.6.114.1-microsoft-standard-WSL2-x86_64-with-glibc2.43
Compilador	gcc (Debian 16.2.0-1) 16.2.0 (C11, -O2)
MPI	Open MPI v5.0.10
OpenMP	libgomp de GCC (bandera -fopenmp)
Criptografía	OpenSSL 3.6.3 9 Jun 2026 (interfaz EVP)
Procesos y hilos	Siempre 4 núcleos: 4×1, 2×2 o 1×4 (procesos × hilos)

Tabla 1. Entorno en el que se midieron todos los tiempos.

Los programas se compilan con `make build` y se prueban con `make test` (pruebas unitarias en C y pruebas de integración en Python). Los tiempos se miden con `MPI_Wtime` (o un reloj monotónico en la versión secuencial) desde una barrera inicial hasta el final de la búsqueda no incluyen la lectura del archivo, la difusión inicial ni el arranque de `mpirun`.

4. Inciso 1: cifrado y descifrado de archivos

El programa `aes_tool` lee un archivo `.txt`, lo cifra con AES-128-CBC usando la clave numérica recibida con `-k`, y escribe el resultado en otro archivo. Con `decrypt` realiza el proceso inverso con la misma clave.

4.1 Conversión de la clave a 128 bits

AES-128 necesita una clave de 16 bytes. Se conservó la conversión del Laboratorio 03: los primeros 8 bytes son el prefijo fijo `5556472d43433330` (en ASCII, «UVG-CC30») y los últimos 8 bytes son el entero de la clave en orden *big-endian*. Para la clave 123456 = 0x1E240:

```
prefijo   55 56 47 2d 43 43 33 30
entero    00 00 00 00 00 01 e2 40
clave     5556472d4343333000000000001e240
```

La búsqueda recorre los enteros de 0 a $2^{56} - 1$, por lo que el espacio explorado es un subconjunto de 2^{56} claves dentro de las 2^{128} posibles de AES-128.

4.2 Formato del archivo cifrado

El archivo cifrado contiene el IV aleatorio de 16 bytes seguido del texto cifrado con relleno PKCS#7. Un mensaje de 32 bytes produce 32 + 16 (relleno) + 16 (IV) = 64 bytes. Se validan los casos límite: archivo vacío, longitudes múltiplo y no múltiplo de 16, texto UTF-8, archivos inexistentes, claves no numéricas o mayores que $2^{64} - 1$, y que el programa no sobrescriba la entrada ni deje una salida a medias si el descifrado falla. El resultado es compatible con `openssl enc -aes-128-cbc`, lo que se comprueba en las pruebas automáticas.

4.3 Evidencia

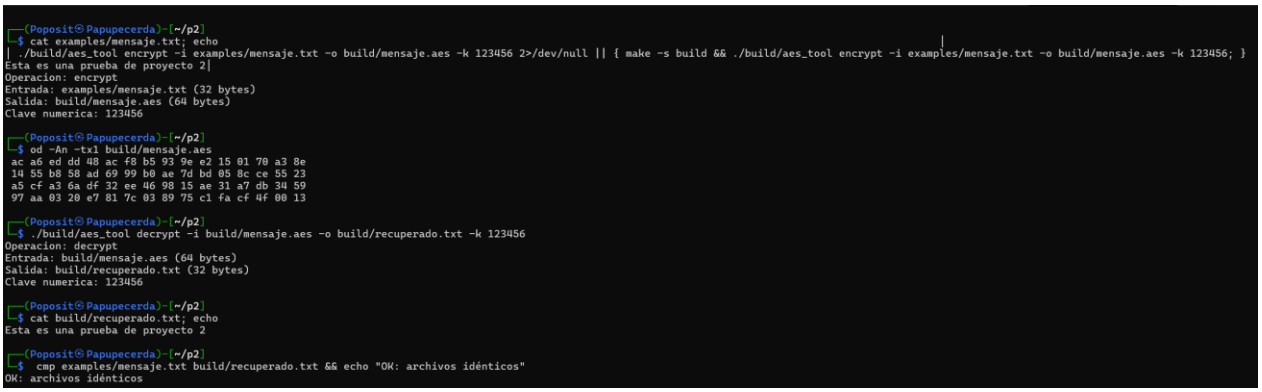


Figura 1. Cifrado y descifrado del mensaje con la clave 123456.

El texto recuperado es idéntico al original (`cmp` no reporta diferencias). El IV cambia en cada cifrado, por lo que dos cifrados del mismo texto con la misma clave producen archivos distintos.

5. Inciso 2: búsqueda de la clave con 4 procesos

5.1 Programa base: estrategia cíclica

La búsqueda recibe solamente el archivo cifrado y la frase de búsqueda; nunca recibe la clave. Para cada candidata construye la clave de 16 bytes, descifra y acepta la candidata si el relleno es válido y el texto contiene *es una prueba de*. La versión MPI conserva el reparto del Laboratorio 03: el rango se divide en bloques de 4096 claves y, en la ronda r, el proceso k prueba el bloque $r \cdot P + k$. Al terminar cada ronda, todos los procesos llaman a `MPI_Allreduce` con la operación mínimo: si alguno encontró la clave, todos la reciben y se detienen. El diagrama de flujo de la sección 7.2 muestra el proceso completo.

5.2 Resultados

Clave	Tiempo	Candidatas probadas	Resultado
a) 123456	4.859 ms	127,553	Encontrada (promedio de 3)
b) 18014398509481984	> 3.0 s (detenida)	83,443,712	Estimado: 7,496 días (≈ 21 años)
c) 18014398509481985	> 3.0 s (detenida)	83,836,928	Estimado: 7,462 días (≈ 20 años)

Tabla 2. Búsqueda con la estrategia cíclica, 4 procesos MPI y 56 bits.

La clave 123456 se encontró en 4.859 ms. Para las dos claves grandes la búsqueda se ejecutó realmente durante 3 segundos con `--seconds 3` y se detuvo sin encontrarlas. Con la tasa medida de 27.8 millones de candidatas por segundo, llegar a 2^{54} requeriría unas 18,014,398,509,494,272 pruebas, alrededor de 7,496 días. Esa cifra es una extrapolación a tasa constante, no un tiempo medido.

5.3 Inciso 2d: ¿por qué varía el tiempo según la clave?

Con la estrategia cíclica los procesos avanzan juntos desde la clave 0 y, después de n rondas, se han probado todas las claves menores que $4096 \cdot 4 \cdot n$. El tiempo es proporcional a la posición de la clave: 123456 está cerca del inicio y basta con 8 rondas (127,553 pruebas, contando las de los otros procesos en la última ronda). Las claves 2^{54} y $2^{54} + 1$ están a la cuarta parte del rango, y antes hay que descartar unas 1.8×10^{16} claves.

Las claves 2^{54} y $2^{54} + 1$ son vecinas: con el reparto cíclico ambas caen en el mismo bloque y en la misma ronda, por lo que tardarían casi lo mismo. La diferencia entre ellas solo se nota en una estrategia en la que alguna empiece justo en 2^{54}, como la del inciso 3. En resumen, el tiempo de la fuerza bruta no depende del valor «grande» o «pequeño» de la clave, sino de cuántas candidatas se prueban antes de llegar a ella con el orden de recorrido elegido.

```
$ mpirun -np 4 --oversubscribe ./build/search_mpi -i build/k123456.aes -b 56 -s cyclic
Estado: encontrada
Clave encontrada: 123456
Mensaje: Esta es una prueba de proyecto 2
Ejecucion: MPI
Procesos: 4
Hilos por proceso: 1
Estrategia: cyclic
Bits: 56
Tamano de chunk: 4096
Candidatas probadas: 127553
Tiempo: 0.004811927 segundos

$ mpirun -np 4 --oversubscribe ./build/search_mpi -i build/k18014398509481984.aes -b 56 -s cyclic --seconds 3
Estado: limite_tiempo
Ejecucion: MPI
Procesos: 4
Hilos por proceso: 1
Estrategia: cyclic
Bits: 56
Tamano de chunk: 4096
Candidatas probadas: 82477056
Tiempo: 3.000025949 segundos
codigo de salida: 3

$ mpirun -np 4 --oversubscribe ./build/search_mpi -i build/k18014398509481985.aes -b 56 -s cyclic --seconds 3
Estado: limite_tiempo
Ejecucion: MPI
Procesos: 4
Hilos por proceso: 1
Estrategia: cyclic
Bits: 56
Tamano de chunk: 4096
Candidatas probadas: 82558976
Tiempo: 3.000358871 segundos
codigo de salida: 3
```

Figura 2. Búsqueda cíclica con 4 procesos para las tres claves del inciso 2.

6. Inciso 3: estrategia alternativa híbrida MPI + OpenMP

6.1 Descripción

La estrategia tiene dos niveles de paralelismo:

- **Nivel 1 (MPI, memoria distribuida):** cada proceso recibe un tramo contiguo del rango. Con $N = 2^{56}$ claves y P procesos, el proceso k recorre [k·N/P, (k+1)·N/P); si N no es divisible entre P, el residuo se reparte entre los primeros procesos.
- **Nivel 2 (OpenMP, memoria compartida):** cada proceso recorre su tramo en bloques de 4096 claves, y cada bloque se divide entre sus hilos con `#pragma omp parallel for` y planificación dinámica en porciones de 64 claves.

Cada hilo tiene su propio contexto de descifrado (`EVP_CIPHER_CTX`) y su propio buffer, de modo que no comparten estado. Si un hilo encuentra la clave, activa una bandera compartida con una escritura atómica y los demás hilos del proceso saltan las candidatas restantes del bloque. Al final de cada bloque solo el hilo principal llama a `MPI_Allreduce` (modo `MPI_THREAD_FUNNELED`) y, si algún proceso halló la clave, todos terminan.

6.2 Justificación

- **Orden de exploración:** con 4 procesos, el proceso 1 empieza exactamente en 2^{54} = 18014398509481984. Las dos claves grandes quedan al inicio de su tramo y se prueban en la primera ronda.

- **Menos comunicación entre procesos:** con más hilos y menos procesos, cada `MPI_Allreduce` coordina menos participantes; la coordinación interna ocurre en memoria compartida.
- **Balance de carga y parada temprana:** la planificación dinámica evita hilos ociosos y la bandera atómica detiene a los hilos de un proceso sin esperar el fin del bloque.
- **Escalabilidad:** en un clúster, MPI reparte tramos entre máquinas y OpenMP aprovecha todos los núcleos de cada máquina.

6.3 Comparación de tiempos

Clave	Bits	Secuencial (ms)	Cíclica 4×1 (ms)	Híbrida 4×1 (ms)	Híbrida 2×2 (ms)	Híbrida 1×4 (ms)
123,456	56	17.104	4.859	19.511	9.987	5.107
262,144	20	36.291	11.796	0.637	19.836	10.063
262,145	20	37.262	10.552	0.628	19.692	9.901
18,014,398,509,481,984	56	≈ 28,675 d*	≈ 7,496 d*	0.589	≈ 14,920 d*	≈ 7,419 d*
18,014,398,509,481,984	56	≈ 28,675 d*	≈ 7,462 d*	0.602	≈ 14,993 d*	≈ 7,555 d*

*Tabla 3. Tiempo de búsqueda en milisegundos (promedio de 3 corridas). *Días estimados a partir de una corrida real de 3 s.*

Las claves 2^{18} y $2^{18} + 1$ sobre un rango de 20 bits son una versión reducida del mismo caso: están al inicio del segundo cuarto del rango y permiten medir las cinco configuraciones hasta el final. Los tiempos individuales de cada corrida están en el Anexo 2.

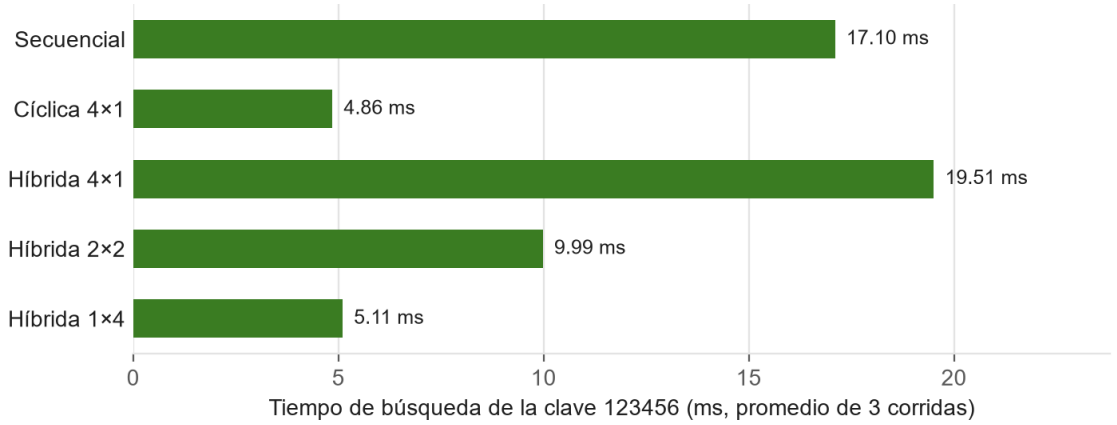


Figura 3. Tiempo para encontrar la clave 123456 con cada configuración.

Con 4 procesos, la estrategia híbrida encontró las dos claves grandes en 0.589 y 0.602 ms, probando 12,289 y 12,290 candidatas: el proceso 1 acierta en su primera o segunda prueba y los otros tres completan su bloque de 4096 antes de la parada. La estrategia cíclica necesitaría unos 7,496 días para las mismas claves.

Para 123456 ocurre lo contrario: la híbrida 4×1 tardó 19.511 ms frente a 4.859 ms de la cíclica. La clave está en el tramo del proceso 0, que la alcanza en su ronda 31; mientras tanto los otros tres procesos prueban claves de sus tramos que no sirven (504,385

pruebas en total). Con 2×2 (9.987 ms) los tramos son dos y la mitad de las pruebas se desperdicia; con 1×4 (5.107 ms) los cuatro hilos recorren el rango desde 0 y el tiempo es casi igual al de la cíclica.

6.4 Tasa de prueba y speedup

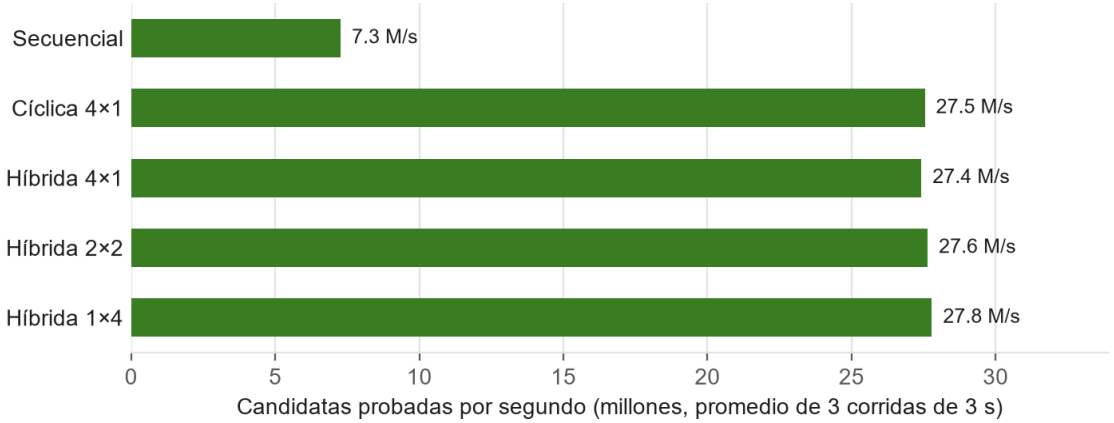


Figura 4. Candidatas probadas por segundo; corridas de 3 s con una clave fuera del rango.

Configuración	Millones de candidatas/s	Speedup de tasa	Eficiencia
Secuencial	7.27	1.00	1.00
Cíclica 4×1	27.54	3.79	0.95
Híbrida 4×1	27.43	3.77	0.94
Híbrida 2×2	27.64	3.80	0.95
Híbrida 1×4	27.79	3.82	0.96

Tabla 4. Tasa de prueba medida (promedio de 3 corridas de 3 s) y speedup respecto a la secuencial.

Las cuatro configuraciones paralelas prueban la misma cantidad de claves por segundo (27.4 a 27.8 millones), alrededor de 3.8 veces la secuencial. Con los mismos 4 núcleos, cambiar procesos por hilos no aumenta la capacidad de cómputo: la ganancia por comunicar menos es pequeña porque cada `MPI_Allreduce` ocurre cada 4096 claves, unos 0.6 ms de trabajo por proceso.

Clave	S Cíclica 4×1	S Híbrida 4×1	S Híbrida 2×2	S Híbrida 1×4
123,456	3.52	0.88	1.71	3.35
262,144	3.08	56.95	1.83	3.61
262,145	3.53	59.29	1.89	3.76
18,014,398,509,481,984	≈ 3.83*	4.2×10^{12}	≈ 1.92*	≈ 3.86*

*Tabla 5. Speedup de tiempo S = T_secuencial / T_paralelo. *Calculado con tiempos estimados.*

El *speedup* de tiempo es casi 4 (lineal) cuando la estrategia prueba aproximadamente las mismas claves que la secuencial: cíclica 4×1 e híbrida 1×4 con 123456. Es superlineal (de miles o millones) cuando la clave queda al inicio de un tramo, y menor que 1 cuando los tramos alejan a la mayoría de los procesos de la clave.

6.5 Ventajas y limitaciones

Estrategia	Ventajas	Limitaciones
Cíclica (inciso 2)	Todos los procesos avanzan juntos desde 0; el tiempo crece con la posición de la clave y nunca prueba muchas más claves que la secuencial. Sencilla de implementar.	Las claves lejanas del inicio, como 2^{54}, son inalcanzables en la práctica. Solo usa MPI: cada núcleo es un proceso y todos participan en cada `MPI_Allreduce`.
Híbrida (inciso 3)	Explora P puntos del rango a la vez: encuentra al instante las claves al inicio de un tramo. Usa memoria compartida dentro de cada nodo y escala a clústeres con nodos multinúcleo.	Su ventaja depende de dónde está la clave: para 123456 fue unas 4 veces más lenta con 4×1. Los tramos dependen del número de procesos: con 2×2 o 1×4 las claves grandes vuelven a quedar lejos. No aumenta las claves por segundo con los mismos núcleos.

Tabla 6. Comparación cualitativa de las estrategias.

7. Inciso 4: funcionamiento de las estrategias

7.1 Reparto de las claves

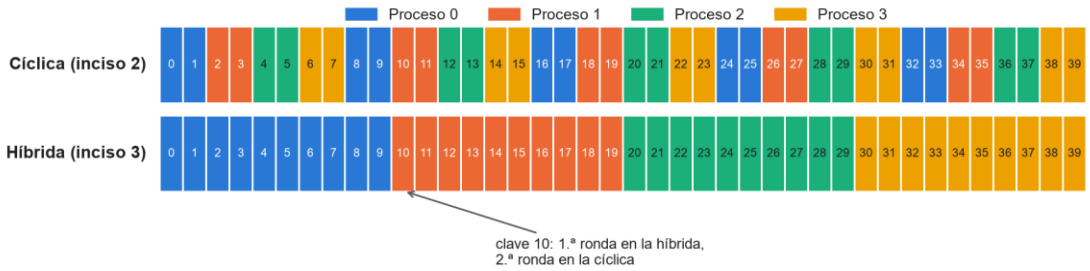


Figura 5. Reparto de 40 claves en bloques de 2 entre 4 procesos (ejemplo reducido).

En la cíclica los bloques se asignan por turnos y los procesos avanzan juntos desde 0. En la híbrida cada proceso recibe un sector consecutivo. En el programa real los bloques son de 4096 claves y el rango tiene 2^{56}; con 4 procesos los sectores empiezan en 0, 2^{54}, $2 \cdot 2^{54}$ y $3 \cdot 2^{54}$.

Proceso	Tramo (extremo superior exclusivo)	Primera clave
0	$[0, 2^{54})$	0
1	$[2^{54}, 2 \cdot 2^{54})$	18,014,398,509,481,984
2	$[2 \cdot 2^{54}, 3 \cdot 2^{54})$	36,028,797,018,963,968
3	$[3 \cdot 2^{54}, 4 \cdot 2^{54})$	54,043,195,528,445,952

Tabla 7. Tramos de la estrategia híbrida con 4 procesos y 56 bits.

7.2 Diagramas de flujo

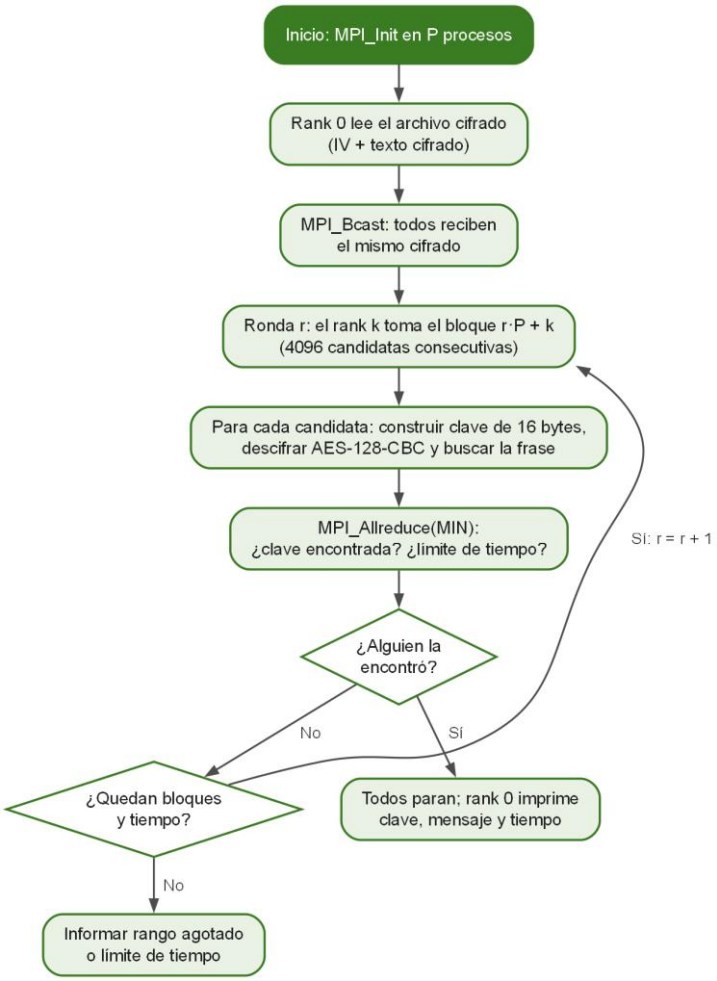


Figura 6. Flujo de la estrategia base (cíclica).

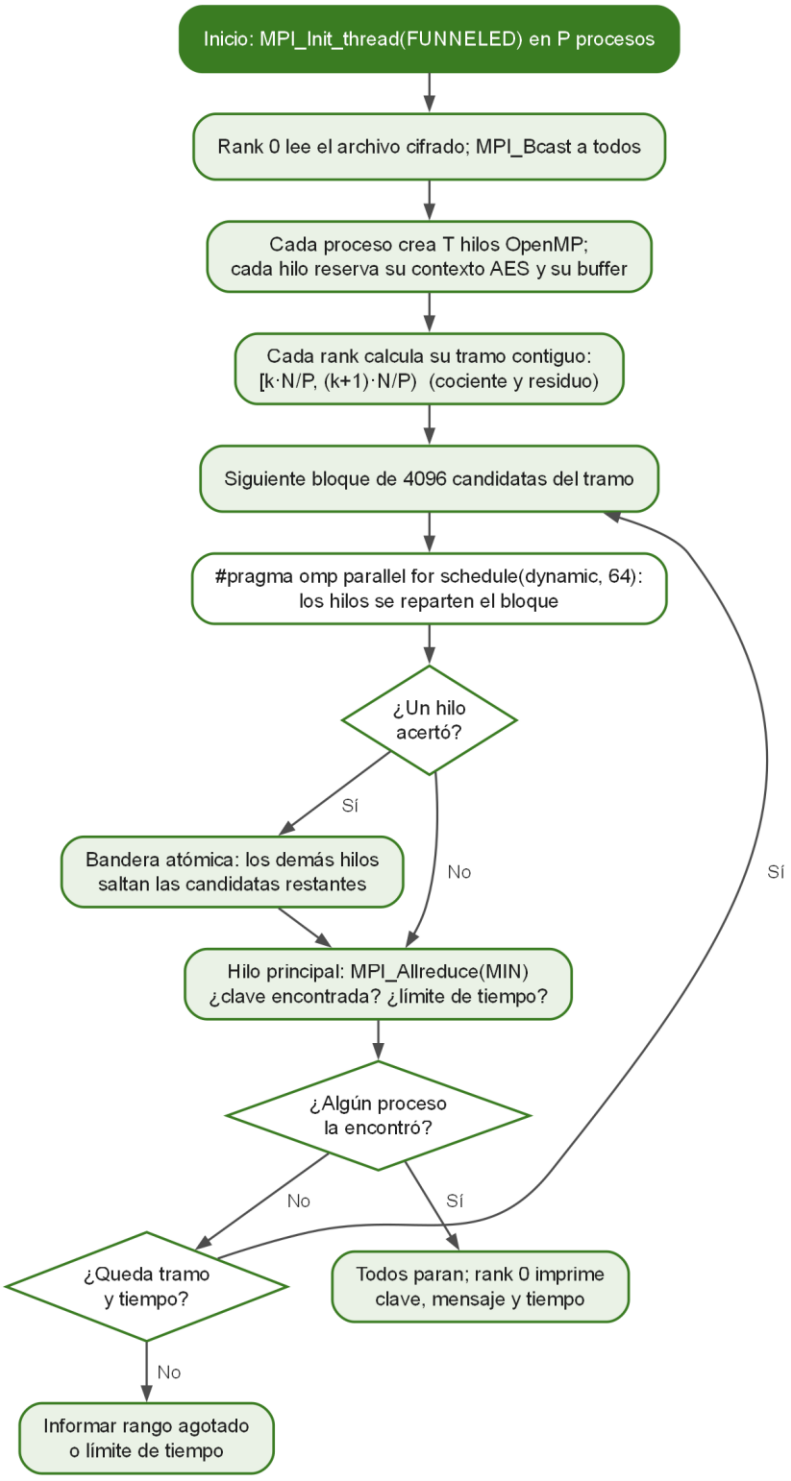


Figura 7. Flujo de la estrategia alternativa (híbrida MPI + OpenMP).

7.3 Pseudocódigo de la estrategia híbrida

```
MPI_Init_thread(FUNNELED)
si rank = 0: leer archivo cifrado
MPI_Bcast(tamaño); MPI_Bcast(IV + cifrado)
en paralelo (T hilos): cada hilo crea su contexto AES y su buffer
inicio, cantidad ← tramo contiguo del rank (cociente y residuo)
para ronda = 0, 1, 2, ...:
    [a, b) ← siguiente bloque de 4096 claves del tramo (vacío si se agotó)
    encontrada ← ∞;  parar ← falso
    omp parallel for schedule(dynamic, 64) sobre clave en [a, b):
        si atomic_leer(parar): continuar
        texto ← descifrar(cifrado, clave_128(clave), contexto del hilo)
        si relleno válido y texto contiene frase:
            critical: encontrada ← min(encontrada, clave)
            atomic_escribir(parar, verdadero)
    global ← MPI_Allreduce(MIN, [encontrada, dentro_del_tiempo])   // solo hilo
principal
    si global.encontrada < ∞ o se acabó el tiempo: salir
MPI_Reduce(MAX, tiempo); MPI_Reduce(SUM, pruebas)
si rank = 0: imprimir clave, mensaje, pruebas y tiempo
MPI_Finalize
```

7.4 Comunicación cuando se encuentra la clave

Hay dos niveles de aviso. Dentro de un proceso, el hilo que acierta escribe la bandera con `omp atomic write`; los demás la leen con `omp atomic read` antes de cada candidata y saltan el resto del bloque. Si dos hilos aciertan en el mismo bloque, una sección crítica conserva la menor clave. Entre procesos, al terminar el bloque, el hilo principal de cada proceso aporta a `MPI_Allreduce` el par {clave encontrada o $2^{64} - 1$, 1 si sigue dentro del tiempo}. La operación mínimo entrega a todos la menor clave encontrada y un 0 si alguno superó el límite de tiempo. Los procesos que ya agotaron su tramo siguen llamando a `MPI_Allreduce` con un bloque vacío, porque una operación colectiva requiere a todos los procesos. Si la clave es válida, todos salen del ciclo, se reducen los tiempos (máximo) y las pruebas (suma) en el rank 0, y este imprime el resultado.

8. Discusión

8.1 Retos y soluciones

- **Falsas coincidencias:** un relleno PKCS#7 válido aparece por azar en aproximadamente 1 de cada 256 claves equivocadas. Se exige además que el texto contenga la frase completa, lo que hace improbable aceptar una clave incorrecta.
- **Parada colectiva sin bloqueos:** si un proceso sale del ciclo antes que los demás, los otros quedan esperando en `MPI_Allreduce`. Se hizo que todos los procesos participen en cada ronda, incluso con un bloque vacío, y se probó con 1, 3 y 5 procesos y rangos que no se dividen exactamente.
- **Claves inalcanzables:** las búsquedas cíclicas de 2^{54} no terminan en un tiempo razonable. Se agregó la opción `--seconds`, con código de salida propio, para medir la tasa real y extrapolar sin confundir un límite de tiempo con un rango agotado.
- **False sharing entre hilos:** en la primera versión, los contextos AES y los buffers de todos los hilos se reservaban seguidos desde el hilo principal. La configuración

1×4 probaba solo unos 18 millones de claves por segundo, frente a 27 millones de 4×1, porque los hilos escribían en las mismas líneas de caché. Se movió la reserva dentro de la región paralela para que cada hilo use su propia arena de memoria, y la tasa se igualó.

- **Entorno WSL:** con el ejecutable ubicado en el disco de Windows (`/mnt/c`), `mpirun` se quedaba colgado de vez en cuando al terminar con un código distinto de 0, incluso con un programa MPI mínimo. Las pruebas y mediciones se ejecutaron desde el sistema de archivos de Linux.

8.2 Lo aprendido

La distribución del trabajo importa tanto como la cantidad de núcleos. Con la misma tasa de prueba, el tiempo para encontrar una clave varió de menos de un milisegundo a miles de años según el orden de exploración. También se comprobó que pasar de procesos a hilos con los mismos núcleos no acelera una tarea que casi no se comunica: el beneficio del modelo híbrido aparece al escalar a más nodos o cuando la comunicación entre procesos es frecuente. Por último, los detalles de memoria (contextos separados por hilo, *false sharing*) son decisivos para que los hilos rindan igual que los procesos.

9. Conclusiones

1. La estrategia cíclica del inciso 2 encontró la clave 123456 en 4.859 ms con 4 procesos (speedup 3.52 frente a la secuencial), pero las claves 2^{54} y $2^{54} + 1$ requerirían unos 21 años, porque están a la cuarta parte del recorrido.
2. El tiempo de una búsqueda por fuerza bruta depende de cuántas claves se prueban antes de llegar a la correcta. La misma clave puede ser instantánea o inalcanzable según cómo se repartan las candidatas.
3. La estrategia híbrida con 4 procesos encontró las dos claves grandes en menos de 1 ms (12,289 y 12,290 pruebas), porque una de ellas es el inicio exacto de un tramo. Para claves bajas como 123456 fue unas 4 veces más lenta.
4. Con los mismos 4 núcleos, las configuraciones 4×1, 2×2 y 1×4 prueban unos 28 millones de claves por segundo: OpenMP no agrega capacidad, pero reduce la comunicación entre procesos y permite aprovechar nodos multinúcleo.

10. Recomendaciones

- Asignar a cada proceso un tramo contiguo solo cuando se pueda aprovechar con muchos procesos (varios nodos), y usar hilos para los núcleos de cada nodo; así se conservan muchos puntos de inicio sin perder el beneficio de la memoria compartida.
- Combinar ambas ideas: intercalar bloques pequeños dentro de cada tramo para que las claves bajas no queden relegadas a un solo proceso.
- Para medir el impacto real de MPI frente a OpenMP, repetir las pruebas en un clúster con varias máquinas, donde la comunicación por red sí es costosa.
- Medir siempre las búsquedas largas con un límite de tiempo y reportar la tasa de prueba, además del tiempo, para no confundir estimaciones con mediciones.

11. Referencias

[1] National Institute of Standards and Technology. Advanced Encryption Standard (AES), FIPS 197 (actualizado), NIST, mayo de 2023. https://doi.org/10.6028/NIST.FIPS.197-upd1

[2] M. Dworkin. Recommendation for Block Cipher Modes of Operation: Methods and Techniques, NIST SP 800-38A, NIST, diciembre de 2001. https://doi.org/10.6028/NIST.SP.800-38A

[3] Message Passing Interface Forum. MPI: A Message-Passing Interface Standard, versión 4.0, junio de 2021. https://www.mpi-forum.org/docs/mpi-4.0/mpi40-report.pdf

[4] OpenMP Architecture Review Board. OpenMP Application Programming Interface, versión 5.2, noviembre de 2021. https://www.openmp.org/spec-html/5.2/openmp.html

[5] P. S. Pacheco y M. Malensek. An Introduction to Parallel Programming, 2.ª ed., Morgan Kaufmann, 2022.

[6] Open MPI Project. MPI_Allreduce, documentación de Open MPI 5. https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Allreduce.3.html

[7] OpenSSL Project. EVP_EncryptInit, documentación de OpenSSL 3.0. https://docs.openssl.org/3.0/man3/EVP_EncryptInit/

Anexo 1. Catálogo de funciones y librerías

A1.1 Funciones del programa

Función (archivo)	Entradas	Salidas	Propósito y funcionamiento
`parse_u64` (common.c)	Texto	1/0; entero de 64 bits	Valida que el texto sea un entero decimal sin signo y sin desbordamiento (rechaza signos, espacios y letras).
`make_key` (common.c)	Candidata de 64 bits	Clave de 16 bytes	Copia el prefijo UVG-CC30 y escribe el entero en big-endian en los últimos 8 bytes.
`read_file` / `write_file` (common.c)	Ruta; datos	Buffer; 1/0	Leen el archivo completo en memoria (con límite inferior a 2 GiB por las interfaces de EVP y MPI) o lo escriben y comprueban fwrite y fclose.
`same_file` (common.c)	Dos rutas	1 si son el mismo archivo	Evita que la salida sobrescriba la entrada.
`valid_ciphertext` (common.c)	Buffer	1/0	Comprueba que el archivo tenga IV y una cantidad entera de bloques de 16 bytes.
`encrypt_buffer` (common.c)	Texto plano, clave	IV + cifrado	Genera un IV aleatorio con RAND_bytes y cifra con AES-128-CBC y relleno PKCS#7 mediante EVP.
`new_decrypt_context` (common.c)	—	EVP_CIPHER_CTX	Crea un contexto de descifrado reutilizable para muchas candidatas.
`decrypt_candidate` (common.c)	Contexto, cifrado, candidata	1 relleno válido, 0 inválido, −1 error; texto plano	Construye la clave de la candidata, descifra el mensaje completo sin relleno automático y valida el relleno PKCS#7 a mano, de modo que una clave errónea es un rechazo normal y no un error de OpenSSL.
`contains_phrase` (common.c)	Texto plano, frase	1/0	Busca la frase en cualquier posición del texto descifrado comparando bytes con memcmp (el texto puede contener bytes nulos).
`monotonic_seconds` (common.c)	—	Segundos	Reloj monotónico para medir la versión secuencial.
`parse_search_options` (search_support.c)	argc, argv, modo	Opciones; 1/0/2	Lee -i, -p, -b, -c, -s, -t y --seconds, rechaza repetidos y valida rangos.
`search_rounds` / `search_bounds` (search_support.c)	Total, procesos, rank, ronda	Rondas; bloque [a, b)	Calculan el reparto cíclico o por tramos contiguos sin huecos ni solapes.
`search_chunk` (search_support.c)	Contexto, bloque [a, b)	Clave o UINT64_MAX; pruebas	Prueba secuencialmente las candidatas de un bloque.
`search_chunk_threads` (search_mpi.c)	Contextos y buffers por hilo, bloque	Clave mínima o UINT64_MAX; pruebas	Reparte el bloque entre hilos OpenMP con planificación dinámica y parada por bandera atómica.
`print_search_result` (search_support.c)	Resultado, tiempos	Código de salida	Verifica de nuevo la clave encontrada e imprime estado, mensaje, configuración, pruebas y tiempo.
`main` (aes_tool.c)	encrypt/decrypt, -i, -o, -k	Archivo; código 0/2	Herramienta del inciso 1.
`main` (search_seq.c)	Opciones de búsqueda	Código 0/1/2/3	Búsqueda secuencial ascendente desde 0.
`main` (search_mpi.c)	Opciones de búsqueda	Código 0/1/2/3	Búsqueda MPI (cíclica o híbrida): difusión, rondas, Allreduce y reducción final.

Tabla 8. Funciones del proyecto. Códigos de salida: 0 encontrada, 1 rango agotado, 2 error, 3 límite de tiempo.

A1.2 Librerías y rutinas externas

Librería	Rutinas utilizadas	Uso en el proyecto
OpenSSL (libcrypto)	EVP_CIPHER_CTX_new/free, EVP_CIPHER_CTX_set_padding, EVP_EncryptInit_ex, EVP_DecryptInit_ex, EVP_*Update, EVP*_Final_ex, RAND_bytes, OPENSSL_cleanse	AES-128-CBC con relleno PKCS#7 e IV aleatorio.
Open MPI	MPI_Init_thread, MPI_Comm_rank, MPI_Comm_size, MPI_Bcast, MPI_Barrier, MPI_Wtime, MPI_Allreduce, MPI_Reduce, MPI_Abort, MPI_Finalize	Procesos, difusión del cifrado, parada colectiva y medición.
OpenMP (libgomp)	omp parallel, omp parallel for schedule(dynamic, 64), reduction, atomic read/write, critical, omp_get_thread_num, omp_set_num_threads	Hilos dentro de cada proceso en la estrategia híbrida.
Biblioteca estándar de C	fopen, fseek/ftell, fread, fwrite, stat, strtoumax, memcpy, memcmp, clock_gettime	Archivos, conversión de números y tiempo.

Tabla 9. Librerías externas.

Anexo 2. Bitácora de pruebas y speedups

A2.1 Pruebas automáticas

`make test` compila y ejecuta `test_core` (conversión de claves, números inválidos, cobertura exacta de las particiones para 1 a 7 procesos y búsqueda de la frase) y 9 pruebas de integración que ejecutan los binarios reales: ida y vuelta de cifrado para distintas longitudes, compatibilidad con `openssl enc`, entradas inválidas, recuperación de 123456 con todas las estrategias, rango agotado con 1, 3 y 5 procesos, claves en los límites de partición, claves grandes con la híbrida, número de hilos inválido y límite de tiempo.

```
OK: claves, conversion numerica, frase y cobertura de particiones.
test_encrypt_decrypt_round_trip ... ok
test_exhaustion_and_collective_termination ... ok
test_first_last_and_partition_boundary ... ok
test_large_keys_fit_and_hybrid_finds_them ... ok
test_openssl_compatibility_and_random_iv ... ok
test_recover_required_short_key ... ok
test_reject_invalid_input_without_overwriting ... ok
test_reject_invalid_thread_count ... ok
test_time_limit_is_not_reported_as_exhaustion ... ok
Ran 9 tests in 34.774s
OK
```

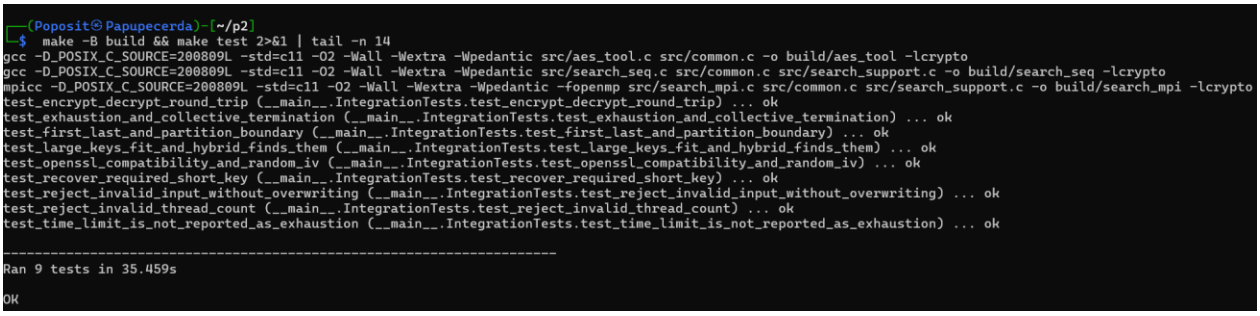


Figura 8. Ejecución de las pruebas automáticas.

A2.2 Corridas individuales

Clave	Configuración	#	Candidatas	Tiempo (s)	Estado
123,456	Secuencial	1	123,457	0.017378	encontrada
123,456	Secuencial	2	123,457	0.016739	encontrada
123,456	Secuencial	3	123,457	0.017196	encontrada
123,456	Cíclica 4×1	1	127,553	0.004785	encontrada
123,456	Cíclica 4×1	2	127,553	0.004804	encontrada
123,456	Cíclica 4×1	3	127,553	0.004987	encontrada
123,456	Híbrida 4×1	1	504,385	0.019920	encontrada
123,456	Híbrida 4×1	2	504,385	0.019398	encontrada
123,456	Híbrida 4×1	3	504,385	0.019215	encontrada
123,456	Híbrida 2×2	1	250,397	0.009622	encontrada
123,456	Híbrida 2×2	2	250,386	0.010501	encontrada
123,456	Híbrida 2×2	3	250,412	0.009839	encontrada
123,456	Híbrida 1×4	1	123,445	0.004547	encontrada
123,456	Híbrida 1×4	2	123,438	0.006249	encontrada

Clave	Configuración	#	Candidatas	Tiempo (s)	Estado
123,456	Híbrida 1×4	3	123,420	0.004525	encontrada
262,144	Secuencial	1	262,145	0.036708	encontrada
262,144	Secuencial	2	262,145	0.036071	encontrada
262,144	Secuencial	3	262,145	0.036095	encontrada
262,144	Cíclica 4×1	1	274,433	0.013165	encontrada
262,144	Cíclica 4×1	2	274,433	0.011585	encontrada
262,144	Cíclica 4×1	3	274,433	0.010639	encontrada
262,144	Híbrida 4×1	1	12,289	0.000631	encontrada
262,144	Híbrida 4×1	2	12,289	0.000648	encontrada
262,144	Híbrida 4×1	3	12,289	0.000632	encontrada
262,144	Híbrida 2×2	1	528,417	0.019407	encontrada
262,144	Híbrida 2×2	2	528,410	0.019720	encontrada
262,144	Híbrida 2×2	3	528,410	0.020380	encontrada
262,144	Híbrida 1×4	1	262,239	0.009537	encontrada
262,144	Híbrida 1×4	2	262,225	0.009909	encontrada
262,144	Híbrida 1×4	3	262,212	0.010743	encontrada
262,145	Secuencial	1	262,146	0.036380	encontrada
262,145	Secuencial	2	262,146	0.038497	encontrada
262,145	Secuencial	3	262,146	0.036908	encontrada
262,145	Cíclica 4×1	1	274,434	0.010407	encontrada
262,145	Cíclica 4×1	2	274,434	0.010598	encontrada
262,145	Cíclica 4×1	3	274,434	0.010652	encontrada
262,145	Híbrida 4×1	1	12,290	0.000684	encontrada
262,145	Híbrida 4×1	2	12,290	0.000608	encontrada
262,145	Híbrida 4×1	3	12,290	0.000593	encontrada
262,145	Híbrida 2×2	1	528,410	0.019202	encontrada
262,145	Híbrida 2×2	2	528,423	0.019236	encontrada
262,145	Híbrida 2×2	3	528,417	0.020638	encontrada
262,145	Híbrida 1×4	1	262,233	0.010517	encontrada
262,145	Híbrida 1×4	2	262,190	0.009607	encontrada
262,145	Híbrida 1×4	3	262,226	0.009578	encontrada
18,014,398,509,481,984	Cíclica 4×1	1	83,443,712	3.000018	límite 3 s
18,014,398,509,481,984	Híbrida 4×1	1	12,289	0.000572	encontrada
18,014,398,509,481,984	Híbrida 4×1	2	12,289	0.000580	encontrada
18,014,398,509,481,984	Híbrida 4×1	3	12,289	0.000616	encontrada
18,014,398,509,481,984	Híbrida 2×2	1	83,853,312	3.000161	límite 3 s
18,014,398,509,481,984	Híbrida 1×4	1	84,307,968	3.000020	límite 3 s
18,014,398,509,481,984	Cíclica 4×1	1	83,836,928	3.000410	límite 3 s
18,014,398,509,481,984	Híbrida 4×1	1	12,290	0.000582	encontrada
18,014,398,509,481,984	Híbrida 4×1	2	12,290	0.000572	encontrada
18,014,398,509,481,984	Híbrida 4×1	3	12,290	0.000651	encontrada
18,014,398,509,481,984	Híbrida 2×2	1	83,443,712	3.000218	límite 3 s
18,014,398,509,481,984	Híbrida 1×4	1	82,796,544	3.000114	límite 3 s

Tabla 10. Todas las corridas de make benchmark (bloques de 4096; 56 bits salvo 2^18 y 2^18+1 con 20 bits).

A2.3 Estimaciones de las búsquedas detenidas

Clave	Configuración	Millones/s medidos	Pruebas previstas	Días estimados
18,014,398,509,481,984	Cíclica 4×1	27.81	18,014,398,509,494,272	7,496
18,014,398,509,481,984	Híbrida 2×2	27.95	36,028,797,018,968,064	14,920
18,014,398,509,481,984	Híbrida 1×4	28.10	18,014,398,509,481,984	7,419
18,014,398,509,481,984	Cíclica 4×1	27.94	18,014,398,509,494,272	7,462
18,014,398,509,481,984	Híbrida 2×2	27.81	36,028,797,018,968,064	14,993
18,014,398,509,481,984	Híbrida 1×4	27.60	18,014,398,509,481,984	7,555

Tabla 11. Extrapolación a tasa constante de las corridas detenidas por --seconds 3.

A2.4 Comandos y salidas registradas

Cada corrida guarda el comando exacto y su salida en `docs/measurements/logs/`. Ejemplos:

```
$ mpirun --oversubscribe -np 4 build/search_mpi -s hybrid -c 4096 -t 1 -i
build/bench_inputs/18014398509481984.aes -b 56
Estado: encontrada
Clave encontrada: 18014398509481984
Mensaje: Esta es una prueba de proyecto 2
Ejecucion: MPI
Procesos: 4
Hilos por proceso: 1
Estrategia: hybrid
Bits: 56
Tamano de chunk: 4096
Candidatas probadas: 12289
Tiempo: 0.000572196 segundos
exit_code=0

$ mpirun --oversubscribe -np 4 build/search_mpi -s cyclic -c 4096 -t 1 -i
build/bench_inputs/18014398509481984.aes -b 56 --seconds 3
Estado: limite_tiempo
Ejecucion: MPI
Procesos: 4
Hilos por proceso: 1
Estrategia: cyclic
Bits: 56
Tamano de chunk: 4096
Candidatas probadas: 83443712
Tiempo: 3.000017838 segundos
    prun:non-zero-exit


exit_code=3
```

Para reproducir: `make build`, `make test`, `make demo`, `make benchmark` y `bash scripts/throughput.sh 3`, desde una copia del proyecto en el sistema de archivos de Linux.

```
$ mpirun -np 4 --oversubscribe ./build/search_mpi -i build/k18014398509481984.aes -b 56 -s hybrid -t 1
Estado: encontrada
Clave encontrada: 18014398509481984
Mensaje: Esta es una prueba de proyecto 2
Ejecucion: MPI
Procesos: 4
Hilos por proceso: 1
Estrategia: hybrid
Bits: 56
Tamano de chunk: 4096
Candidatas probadas: 12289
Tiempo: 0.000560467 segundos

$ mpirun -np 4 --oversubscribe ./build/search_mpi -i build/k18014398509481985.aes -b 56 -s hybrid -t 1
Estado: encontrada
Clave encontrada: 18014398509481985
Mensaje: Esta es una prueba de proyecto 2
Ejecucion: MPI
Procesos: 4
Hilos por proceso: 1
Estrategia: hybrid
Bits: 56
Tamano de chunk: 4096
Candidatas probadas: 12290
Tiempo: 0.000615335 segundos

┌──(Poposit㉿Papupecerda)-[~/p2]
└─$
```

Figura 9. Estrategia híbrida encontrando las dos claves grandes.

```
$ mpirun -np 2 --oversubscribe ./build/search_mpi -i build/k123456.aes -b 56 -s hybrid -t 2
Estado: encontrada
Clave encontrada: 123456
Mensaje: Esta es una prueba de proyecto 2
Ejecucion: MPI
Procesos: 2
Hilos por proceso: 2
Estrategia: hybrid
Bits: 56
Tamano de chunk: 4096
Candidatas probadas: 250412
Tiempo: 0.010196058 segundos

$ mpirun -np 1 --oversubscribe ./build/search_mpi -i build/k123456.aes -b 56 -s hybrid -t 4
Estado: encontrada
Clave encontrada: 123456
Mensaje: Esta es una prueba de proyecto 2
Ejecucion: MPI
Procesos: 1
Hilos por proceso: 4
Estrategia: hybrid
Bits: 56
Tamano de chunk: 4096
Candidatas probadas: 123467
Tiempo: 0.005010907 segundos

┌──(Poposit㉿Papupecerda)-[~/p2]
└─$
```

Figura 10. Configuraciones 2×2 y 1×4 de la estrategia híbrida.